Department of
Computer Science and Engineering

Title: Huffman Coding Algorithm

Algorithm Lab

CSE 206

Green University of Bangladesh

1 Objective(s)

- To define Huffman Coding.
- To understand how Huffman Coding works.
- To implement Huffman Coding algorithm.

2 Problem Analysis

The Huffman Coding Algorithm was proposed by David A. Huffman in 1950. It is a lossless data compression mechanism. It is also known as data compression encoding. It is widely used in image (JPEG or JPG) compression. In this section, we will discuss the Huffman encoding and decoding, and also implement its algorithm in a Java program.

We know that each character is a sequence of 0's and 1's and stores using 8-bits. The mechanism is called fixed-length encoding because each character uses the same number of fixed-bit storage.

Here, a question ascends, is it possible to reduce the amount of space required to store a character?

Yes, it is possible by using variable-length encoding. In this mechanism, we exploit some characters that occur more frequently in comparison to other characters. In this encoding technique, we can represent the same piece of text or string by reducing the number of bits.

2.1 Huffman Encoding

Huffman encoding implements the following steps:

- It assigns a variable-length code to all the given characters.
- The code length of a character depends on how frequently it occurs in the given text or string.
- A character gets the smallest code if it frequently occurs.
- A character gets the largest code if it least occurs.

Huffman coding follows a prefix rule that prevents ambiguity while decoding. The rule also ensures that the code assigned to the character is not treated as a prefix of the code assigned to any other character.

There are the following two major steps involved in Huffman coding:

- First, construct a Huffman tree from the given input string or characters or text.
- Assign, a Huffman code to each character by traversing over the tree.

Let's brief the above two steps.

© Dept. of Computer Science and Engineering, GUB

2.2 Huffman Tree

- **Step 1:** For each character of the node, create a leaf node. The leaf node of a character contains the frequency of that character.
- **Step 2:** Set all the nodes in sorted order according to their frequency.
- **Step 3:** There may exist a condition in which two nodes may have the same frequency. In such a case, do the following:
  1. Create a new internal node.
  2. The frequency of the node will be the sum of the frequency of those two nodes that have the same frequency.
  3. Mark the first node as the left child and another node as the right child of the newly created internal node.
- **Step 4:** Repeat step 2 and 3 until all the node forms a single tree. Thus, we get a Huffman tree.

2.3 Huffman Encoding Example

Suppose, we have to encode string **abracadabra**. Determine the following:

- Huffman code for All the characters.
- Average code length for the given String.
- Length of the encoded string.

2.3.1 (i) Huffman Code for All the Characters

In order to determine the code for each character, first, we construct a **Huffman tree**.

Step 1: Make pairs of characters and their frequencies.

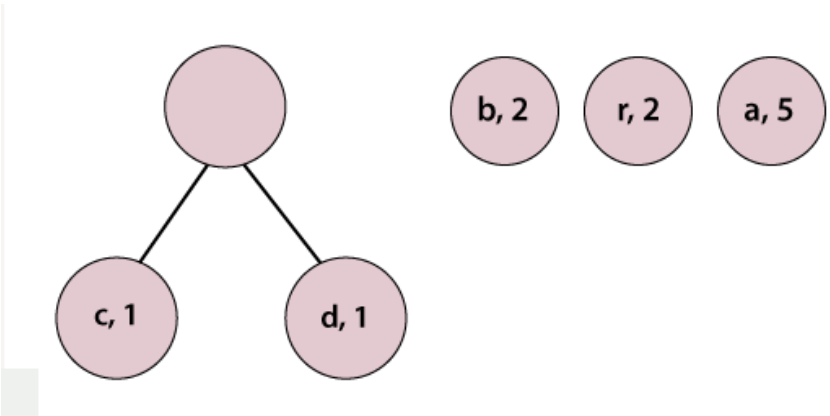
$$(a, 5), (b, 2), (c, 1), (d, 1), (r, 2)$$

Step 2: Sort pairs with respect to frequency, we get:

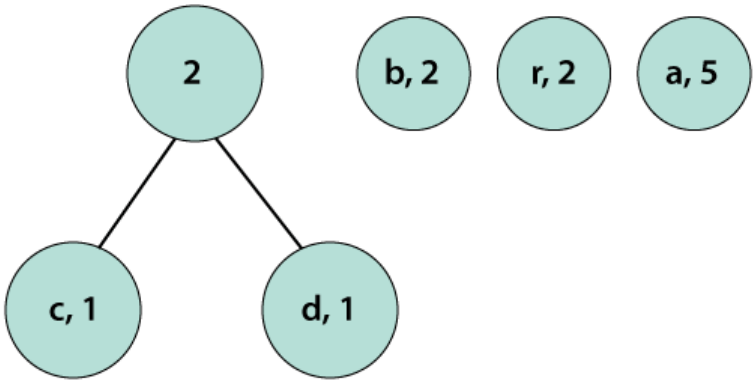
$$(c, 1), (d, 1), (b, 2)(r, 2), (a, 5)$$

© Dept. of Computer Science and Engineering, GUB

Step 3: Pick the first two characters and join them under a parent node.



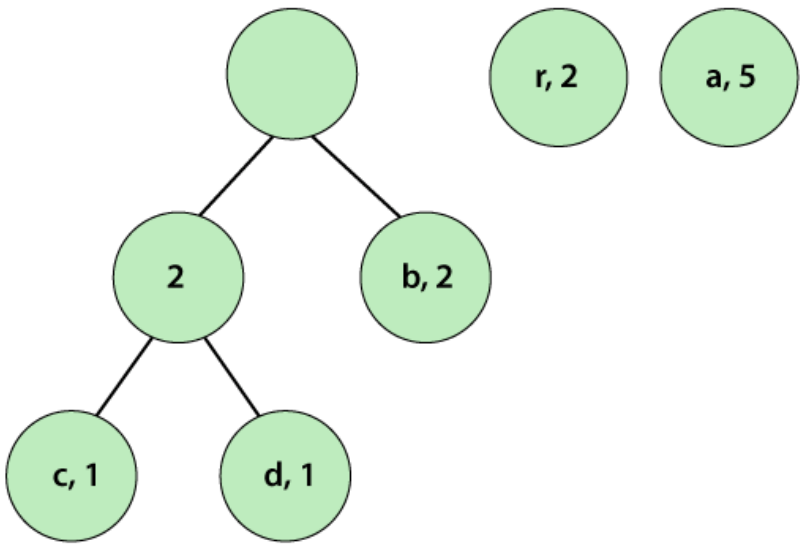
We observe that a parent node does not have a frequency so, we must assign a frequency to it. The parent node frequency will be the sum of its child nodes (left and right) i.e. 1+1=2.



Step 4: Repeat Steps 2 and 3 until, we get a single tree.

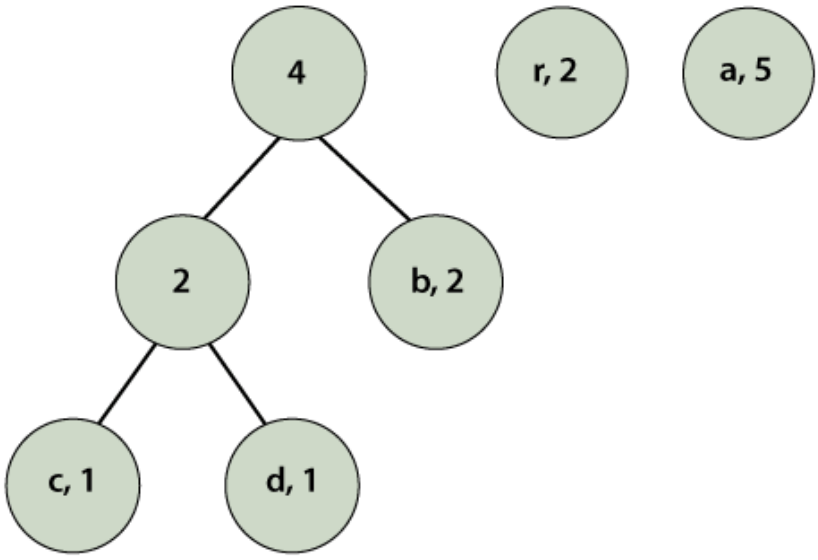
© Dept. of Computer Science and Engineering, GUB

We observe that the pairs are already in a sorted (by step 2) manner. Again, pick the first two pairs and join them.

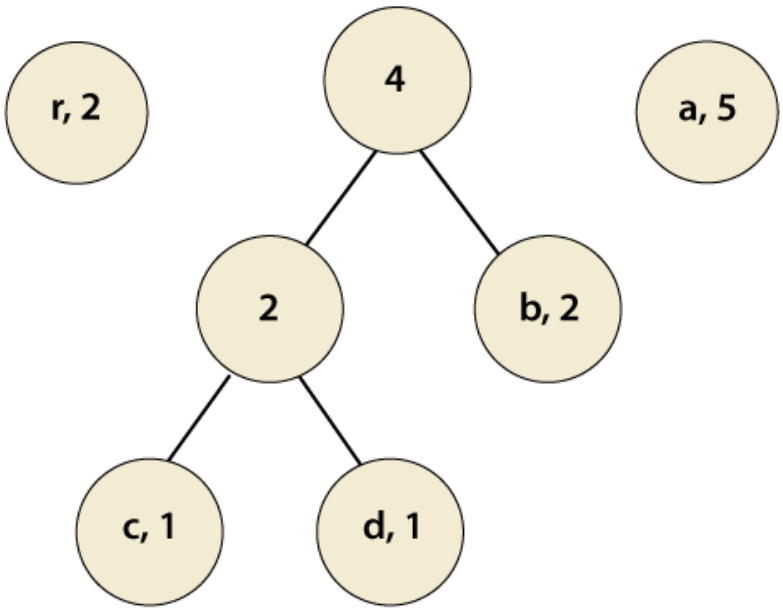


We observe that a parent node does not has a frequency so, we must assign a frequency to it. The parent node frequency will be the sum of its child nodes (left and right) i.e. 2+2=4.

© Dept. of Computer Science and Engineering, GUB

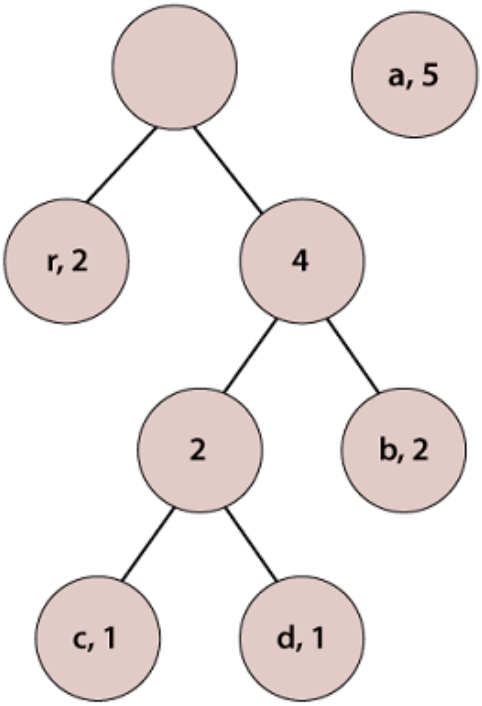


Again, we check if the pairs are in a sorted manner or not. At this step, we need to sort the pairs.

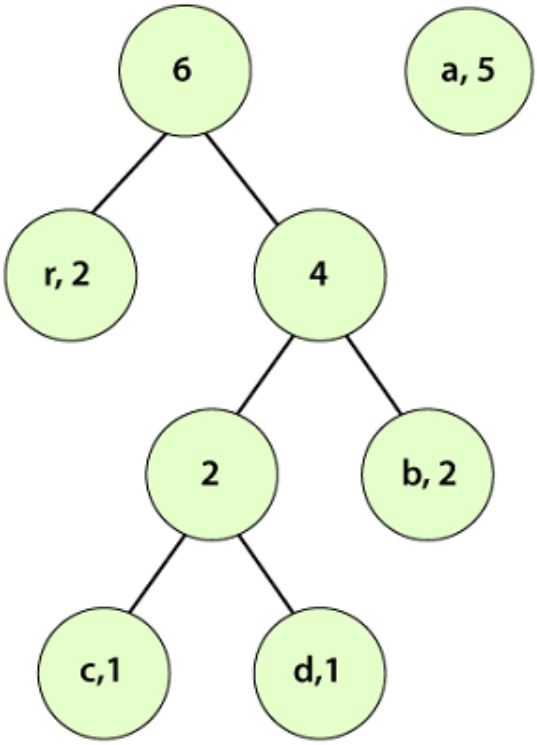


According to step 3, pick the first two pairs and join them, we get:

© Dept. of Computer Science and Engineering, GUB

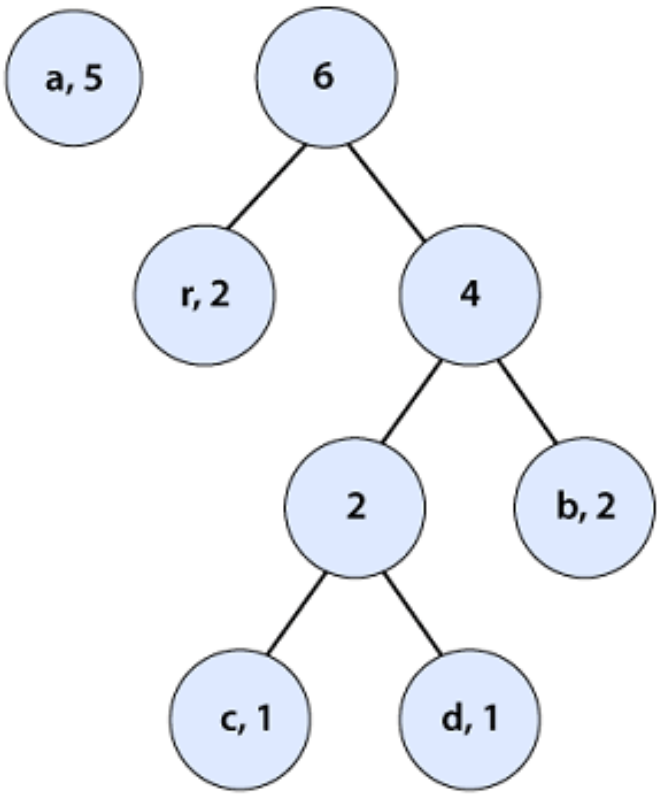


We observe that a parent node does not have a frequency so, we must assign a frequency to it. The parent node frequency will be the sum of its child nodes (left and right) i.e. 2+4=6.



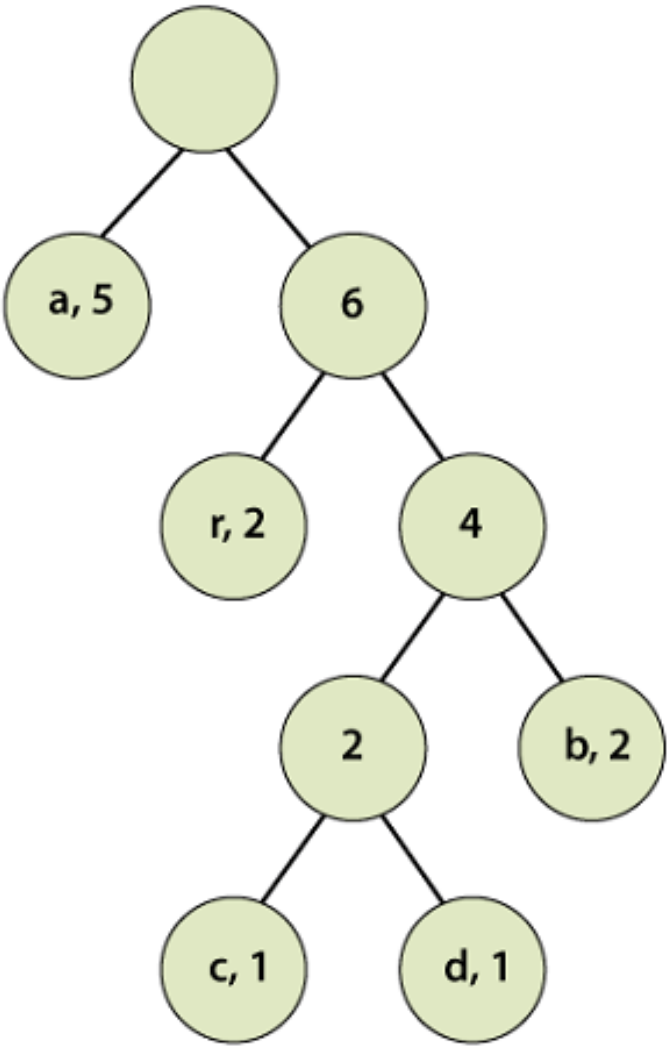
© Dept. of Computer Science and Engineering, GUB

Again, we check if the pairs are in a sorted manner or not. At this step, we need to sort the pairs. After sorting the tree looks like the following:



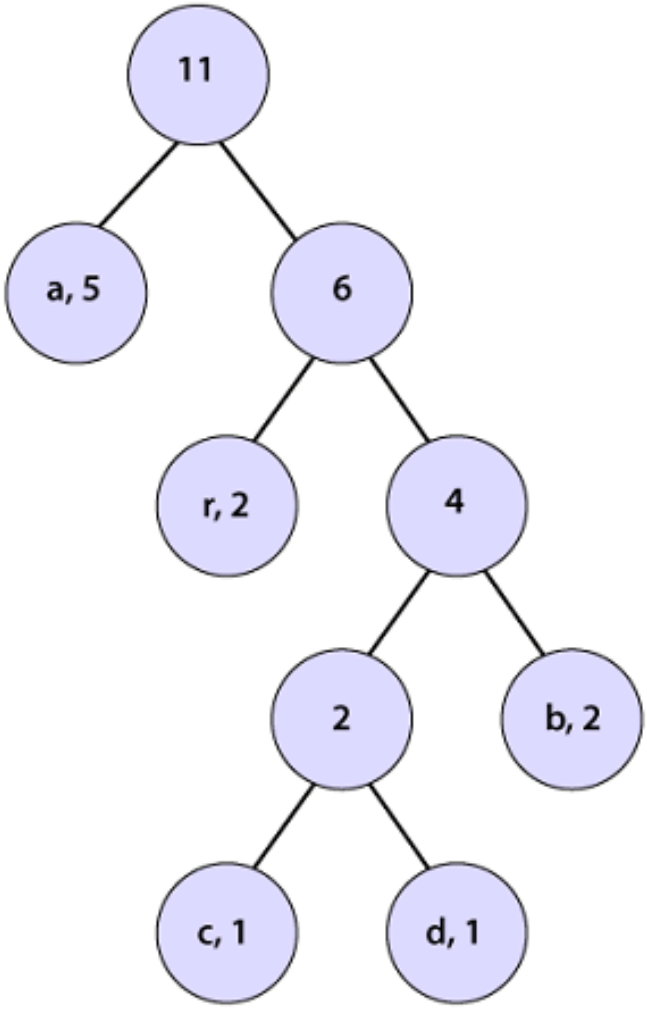
According to step 3, pick the first two pairs and join them, we get:

© Dept. of Computer Science and Engineering, GUB



We observe that a parent node does not have a frequency so, we must assign a frequency to it. The parent node frequency will be the sum of its child nodes (left and right) i.e. 5+6=11.

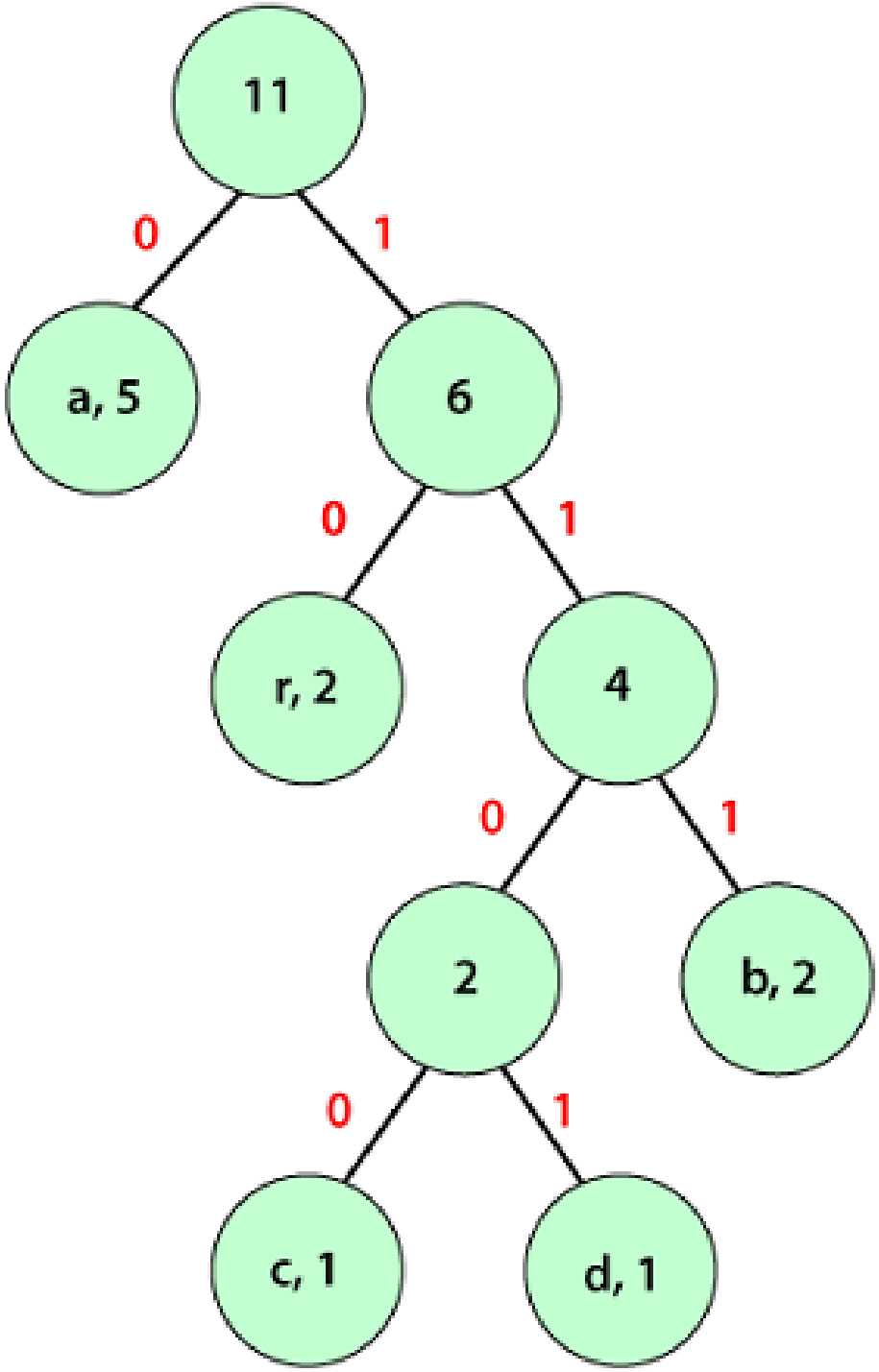
© Dept. of Computer Science and Engineering, GUB



Therefore, we get a single tree.

At last, we will find the code for each character with the help of the above tree. Assign a weight to each edge. Note that each left edge-weighted is 0 and the right edge-weighted is 1.

© Dept. of Computer Science and Engineering, GUB



© Dept. of Computer Science and Engineering, GUB

We observe that input characters are only presented in the leave nodes and the internal nodes have null values. In order to find the Huffman code for each character, traverse over the Huffman tree from the root node to the leaf node of that particular character for which we want to find code. The table describes the code and code length for each character.

Character	Frequency	Code	Code Length
A	5	0	1
B	2	111	3
C	1	1100	4
D	1	1101	4
R	2	10	2

We observe that the most frequent character gets the shortest code length and the less frequent character gets the largest code length.

Now we can encode the string **(abracadabra)** that we have taken above.

0 111 10 0 1100 0 1101 0 111 10 0

© Dept. of Computer Science and Engineering, GUB

2.3.2 (ii) Average Code Length for the String

The average code length of the Huffman tree can be determined by using the formula given below:

$$\text{Average Code Length} = \sum(\text{frequency} \times \text{code length}) / \sum(\text{frequency})$$

= (5 x 1) + (2 x 3) + (1 x 4) + (1 x 4) + (2 x 2) / (5+2+1+1+2)

= 2.09090909

2.3.3 (iii) Length of the Encoded String

The length of the encoded message can be determined by using the following formula:

$$\text{length} = \text{Total number of characters in the text} \times \text{Average code length per character}$$

= 11 x 2.09090909

= 23 bits

© Dept. of Computer Science and Engineering, GUB

3 Algorithm

Algorithm 1: Huffman Coding

Algorithm *Huffman(c)*:
  $n = |c|$
  $Q = c$
  for $i < -1$ *to* $n - 1$ **do**
    $temp = get_node()$
    $left\,[temp]\ get_min(Q)$
    $right\,[temp]\ get_min(Q)$
    $a = left\,[temp]$
    $b = right\,[temp]$
    $F\,[temp] = f[a] + [b]$
    $insert(Q, temp)$
  end
return get_min(0)

4 Implementation

```java
// Huffman Coding in Java

import java.util.PriorityQueue;
import java.util.Comparator;

class HuffmanNode {
  int item;
  char c;
  HuffmanNode left;
  HuffmanNode right;
}

// For comparing the nodes
class ImplementComparator implements Comparator<HuffmanNode> {
  public int compare(HuffmanNode x, HuffmanNode y) {
    return x.item - y.item;
  }
}

// IMplementing the huffman algorithm
public class Main {
  public static void printCode(HuffmanNode root, String s) {
    if (root.left == null && root.right == null && Character.
       isLetter(root.c)) {

      System.out.println(root.c + "    |  " + s);

      return;
```

© Dept. of Computer Science and Engineering, GUB

```
    }
    printCode(root.left, s + "0");
    printCode(root.right, s + "1");
  }

  public static void main(String[] args) {

    int n = 5;
    char[] charArray = { 'A', 'B', 'C', 'D', 'R' };
    int[] charfreq = { 5, 2, 1, 1, 2 };

    PriorityQueue<HuffmanNode> q = new PriorityQueue<HuffmanNode>(n
       , new ImplementComparator());

    for (int i = 0; i < n; i++) {
      HuffmanNode hn = new HuffmanNode();

      hn.c = charArray[i];
      hn.item = charfreq[i];

      hn.left = null;
      hn.right = null;

      q.add(hn);
    }

    HuffmanNode root = null;

    while (q.size() > 1) {

      HuffmanNode x = q.peek();
      q.poll();

      HuffmanNode y = q.peek();
      q.poll();

      HuffmanNode f = new HuffmanNode();

      f.item = x.item + y.item;
      f.c = '-';
      f.left = x;
      f.right = y;
      root = f;

      q.add(f);
    }
    System.out.println(" Char | Code ");
    System.out.println("-------------");
```

© Dept. of Computer Science and Engineering, GUB

```
    printCode(root, "");
  }
}
```

5 Sample Input/Output (Compilation, Debugging & Testing)

Output:

Char | Code

A | 0
B | 10
R | 110
C | 1110
D | 1111

6 Discussion & Conclusion

Huffman coding is a lossless data compression algorithm that assigns variable-length binary codes to symbols based on their frequency of occurrence. The algorithm constructs a Huffman tree to determine the optimal prefix code.

7 Lab Task (Please implement yourself and show the output to the instructor)

Construct a Huffman tree by using these nodes.

Value	A	B	C	D	E	F
Frequency	5	25	7	15	4	12

8 Lab Exercise (Submit as a report)

A file contains the following characters with the frequencies as shown. If Huffman Coding is used for data compression, determine-

1. Huffman Code for each character
2. Average code length
3. Length of Huffman encoded message (in bits)

© Dept. of Computer Science and Engineering, GUB

Characters	A	E	I	O	U	S	T
Frequencies	10	15	12	3	4	13	1

9 Policy

Copying from internet, classmate, seniors, or from any other source is strongly prohibited. 100% marks will be *deducted* if any such copying is detected.

© Dept. of Computer Science and Engineering, GUB